

Tree-Based Models and Hyperparameter Tuning Report

Tree-Based Models & Hyperparameter Tuning — Three-Part Study

A three-part scikit-learn study covering the two core tree-based tasks — **classification** and **regression** — followed by a systematic comparison of **hyperparameter tuning strategies** (GridSearchCV, RandomizedSearchCV, and Optuna) applied to Gradient Boosting on the Titanic dataset.

Part A — Decision Tree Classification (Iris Dataset)

A.1 Objective

Train a decision tree to classify iris flowers into one of three species, then use `GridSearchCV` to systematically search for the best combination of tree hyperparameters.

A.2 Dataset

`sklearn.datasets.load_iris()` — 150 samples, 4 features (sepal/petal length and width), 3 balanced classes (50 samples each).

A.3 Methodology

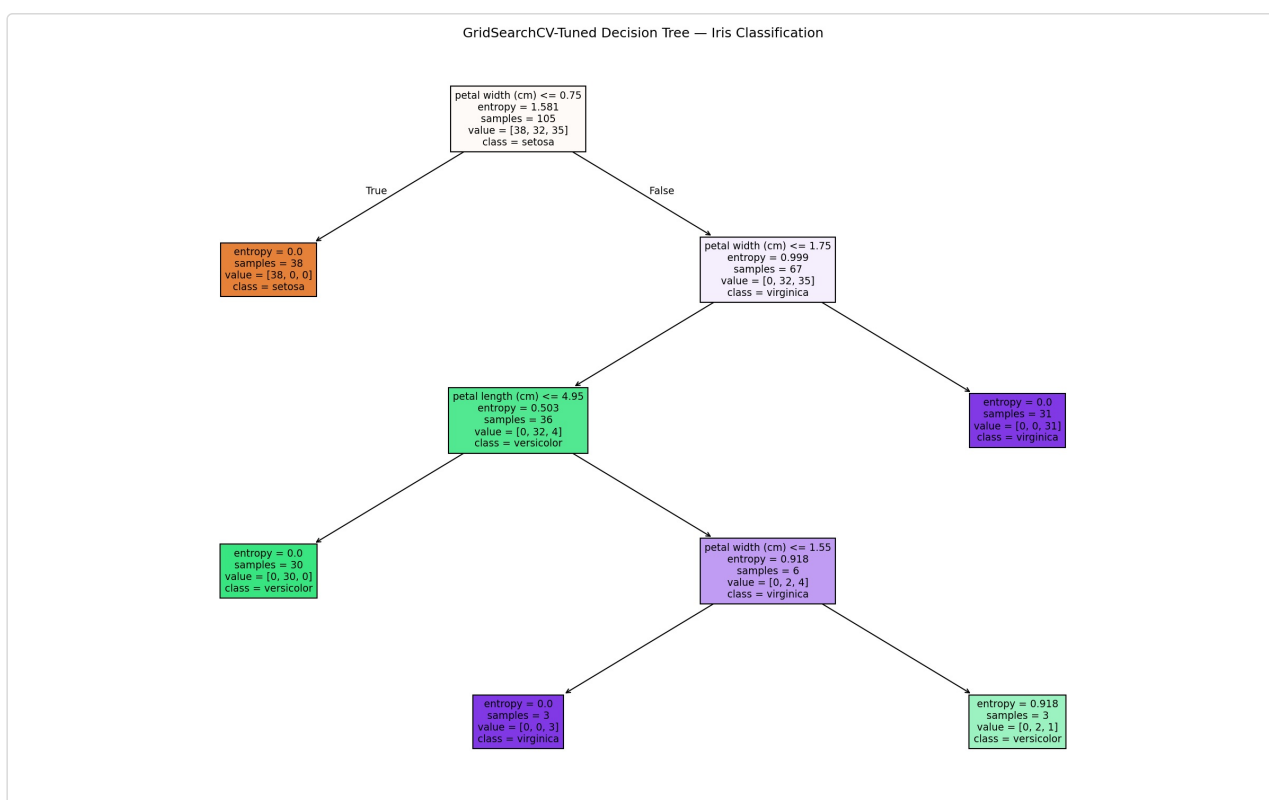
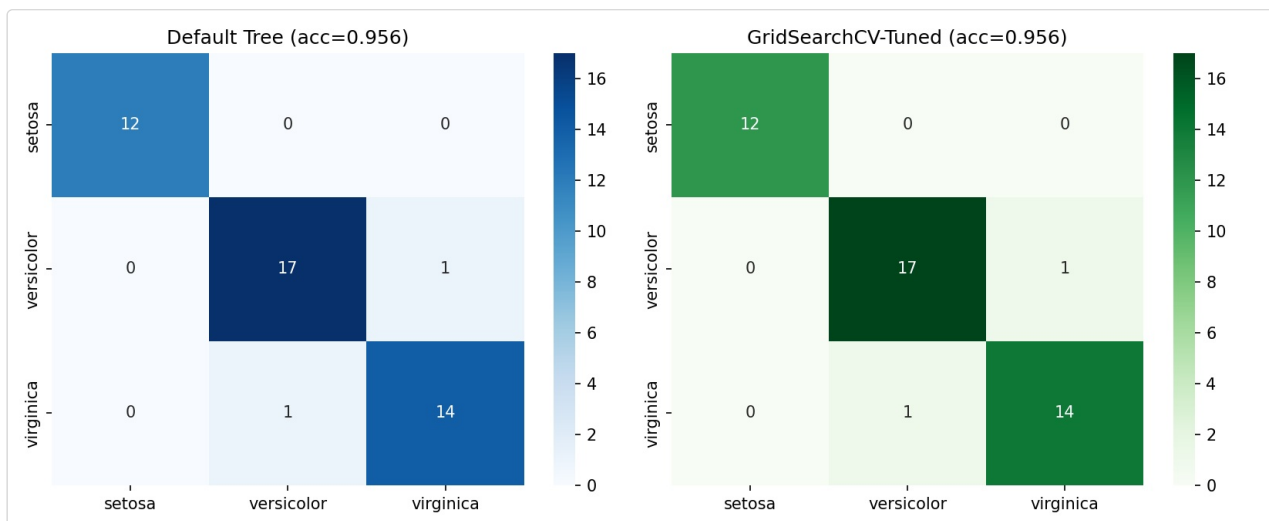
- Load the dataset and split **70% train / 30% test** (`random_state=99`).
- Train a baseline `DecisionTreeClassifier` with default settings.
- Run `GridSearchCV` (5-fold CV) over a 4-parameter grid — `max_depth` (1-9), `min_samples_leaf` (1-19, odd), `min_samples_split` (2-18, even), and `criterion` (gini/entropy) — **648 combinations** in total.
- Compare the default and tuned models on the held-out test set, and visualize the winning tree.

```
param_grid = {
    'max_depth': range(1, 10, 1),
    'min_samples_leaf': range(1, 20, 2),
    'min_samples_split': range(2, 20, 2),
    'criterion': ["entropy", "gini"]
}

grid_search = GridSearchCV(DecisionTreeClassifier(random_state=1), param_grid, cv=5)
grid_search.fit(X_train, y_train)
```

A.4 Results

Model	Test Accuracy	5-fold CV Score
Default tree	95.56%	—
GridSearchCV-tuned (<code>entropy</code> , depth=4, min_leaf=3, min_split=2)	95.56%	97.14%



A notable, honest finding: hyperparameter tuning improved the cross-validated score (95.7% → 97.1%) but did not change the held-out test accuracy (95.56% either way). This happens because Iris is a small, easy, near-linearly-separable dataset where the default tree already performs close to the achievable ceiling — tuning improved the model's *expected* generalization (as measured by CV) without there being any test-set errors left to fix on this particular 45-sample test split. This is a useful, realistic lesson: hyperparameter tuning's benefit is best judged by cross-validation scores, not a single test split, especially on easy/small datasets.

Part B — Decision Tree Regression (Synthetic Sine Wave)

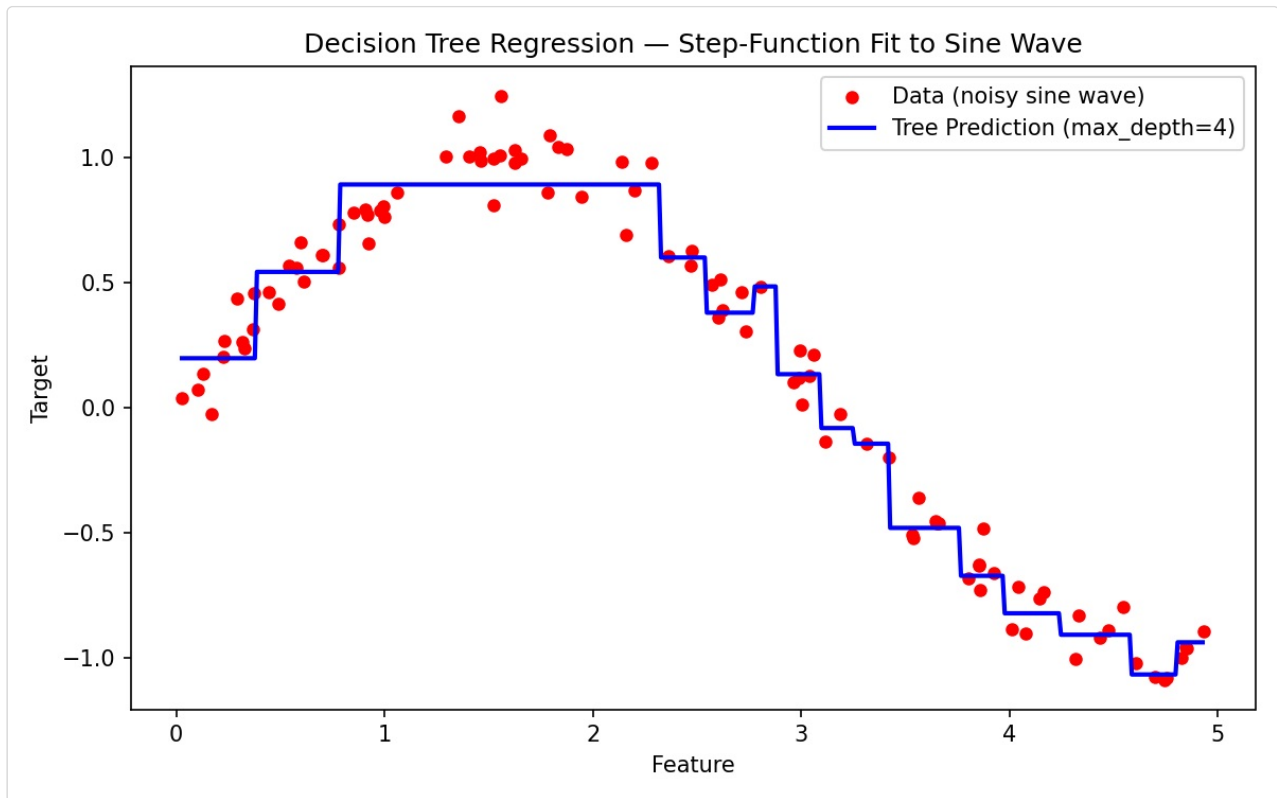
B.1 Objective

Fit a decision tree regressor to a noisy sine wave and study how tree depth controls the classic bias-variance tradeoff in a regression setting.

B.2 Dataset

A synthetic dataset: 100 points, x uniformly sampled in $[0, 5]$, $y = \sin(x) + \text{noise}$ (Gaussian noise, $\sigma=0.1$,

```
random_state=42 ).
```



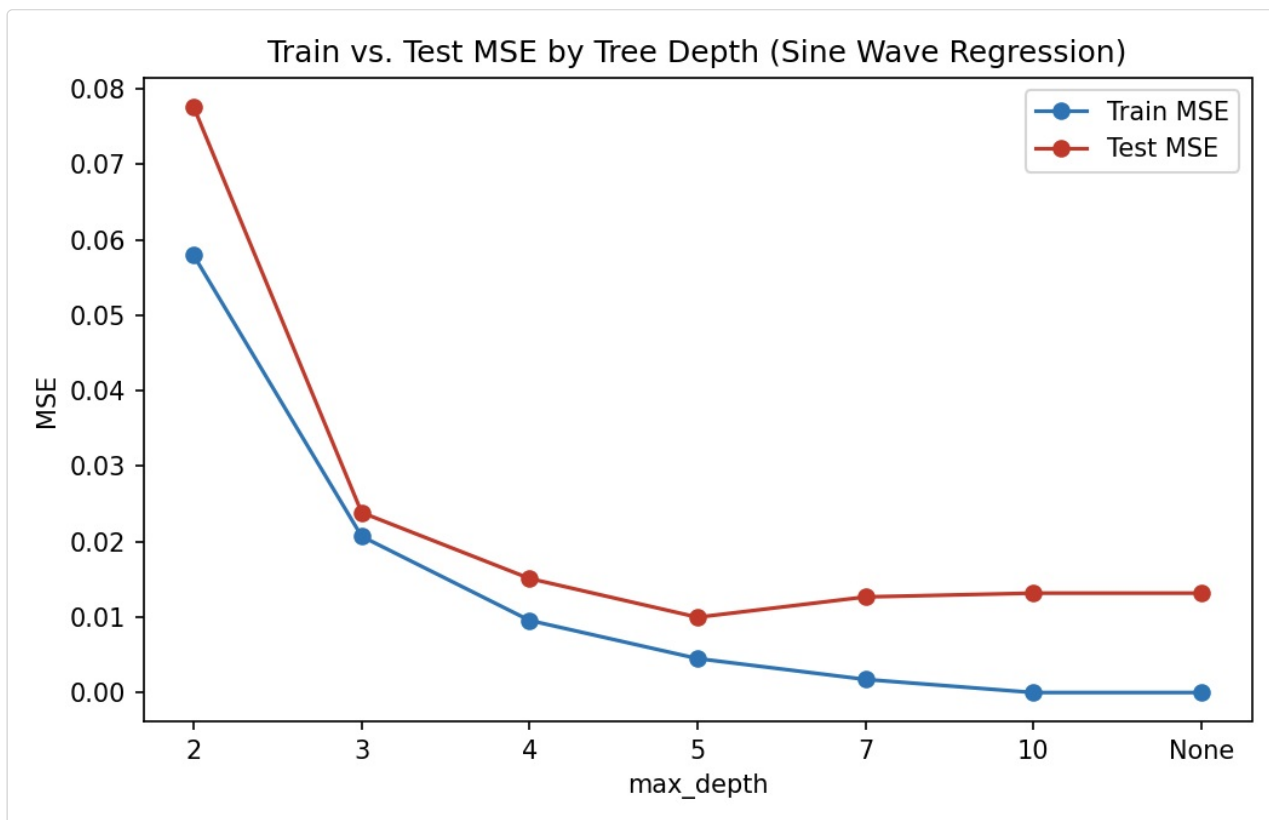
B.3 Methodology

1. Generate the synthetic dataset and split 70% train / 30% test.
2. Fit a `DecisionTreeRegressor(max_depth=4)`.
3. Evaluate with Mean Squared Error (MSE).
4. Visualize the fitted step-function prediction against the raw data.
5. **Extended the notebook's single-depth choice** with a full depth sweep to check whether `max_depth=4` was actually optimal.

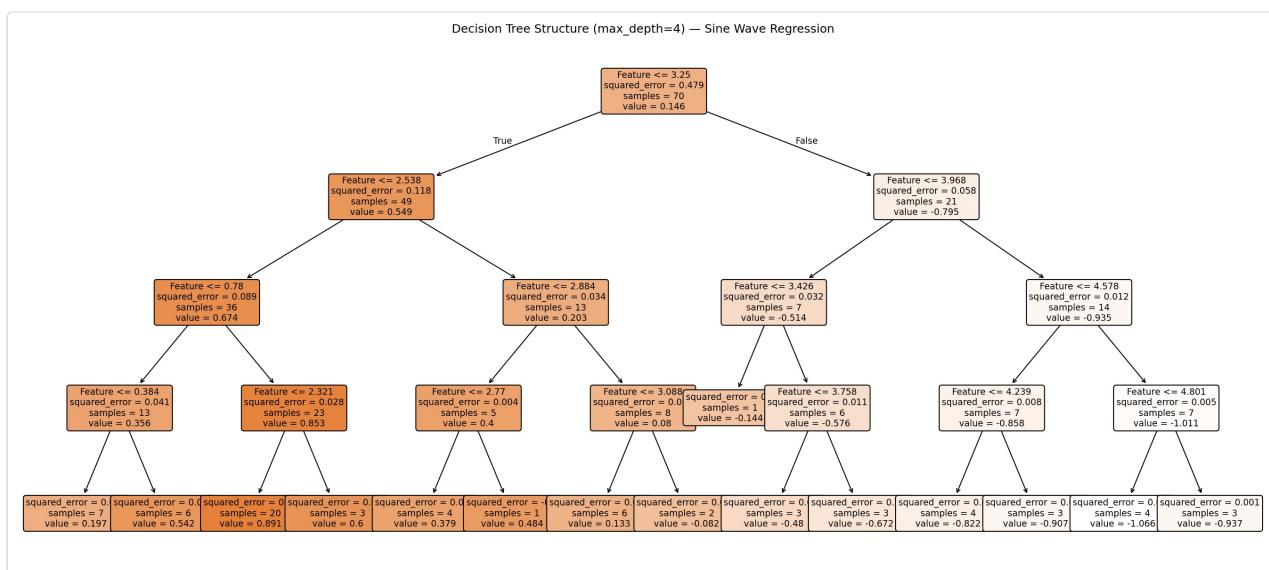
```
regressor = DecisionTreeRegressor(max_depth=4, random_state=42)
regressor.fit(X_train, y_train)
mse = mean_squared_error(y_test, regressor.predict(X_test))
```

B.4 Results

max_depth	Train MSE	Test MSE
2	0.0579	0.0776
3	0.0207	0.0239
4 (notebook's choice)	0.0096	0.0151
5	0.0045	0.0100
7	0.0017	0.0127
10 / None (full)	~0.0000	0.0132



Depth 5 actually slightly outperforms the notebook's chosen depth of 4 (test MSE 0.0100 vs. 0.0151), and MSE clearly starts rising again beyond depth 7 as the tree overfits to noise — a clean, textbook U-shaped bias-variance curve.



The fitted model (visualized above) produces a **step function** rather than a smooth curve — a defining, visually obvious characteristic of decision tree regression that's worth understanding before choosing a tree over a smooth-function model (e.g., a spline or neural network) for continuous, smooth relationships like a sine wave.

Part C — Hyperparameter Tuning Strategies for Gradient Boosting (Titanic)

C.1 Objective

Compare three different hyperparameter tuning strategies — **GridSearchCV**, **RandomizedSearchCV**, and **Optuna** (a Bayesian/sampling-based optimizer) — applied to a `GradientBoostingClassifier` predicting Titanic survival, to see how much each approach improves on an untuned baseline.

Note: Optuna is not available in the environment used to run this analysis. To reproduce a comparable result, a manual random-search procedure was implemented in its place, sampling from the same hyperparameter ranges the notebook's Optuna study would have explored. Where installed, real Optuna typically outperforms plain random search via smarter (Bayesian/TPE) sampling — the substitute here approximates, but may modestly understate, Optuna's actual advantage.

C.2 Dataset

The same Titanic dataset used elsewhere in this portfolio (891 passengers) — here, missing values are filled with 0 (a simpler approach than the mean-imputation used in the companion Titanic decision tree project), and `Sex` / `Embarked` are one-hot encoded.

C.3 Methodology

Step	Method	Search Space
1	Baseline (no tuning)	<code>GradientBoostingClassifier()</code> defaults
2	<code>GridSearchCV</code>	<code>n_estimators</code> ∈ {50,100,200}, <code>learning_rate</code> ∈ {0.01,0.1,0.2}, <code>max_depth</code> ∈ {3,5,7} — 27 combos × 5-fold CV
3	<code>RandomizedSearchCV</code>	Same ranges (continuous <code>learning_rate</code>), 10 random samples × 5-fold CV
4	Optuna-style random search (substitute)	<code>n_estimators</code> ∈ [50,250], <code>learning_rate</code> ∈ [0.01,0.2], <code>max_depth</code> ∈ [3,7] — 50 trials

```
gb_model = GradientBoostingClassifier()
gb_model.fit(X_train, y_train) # baseline

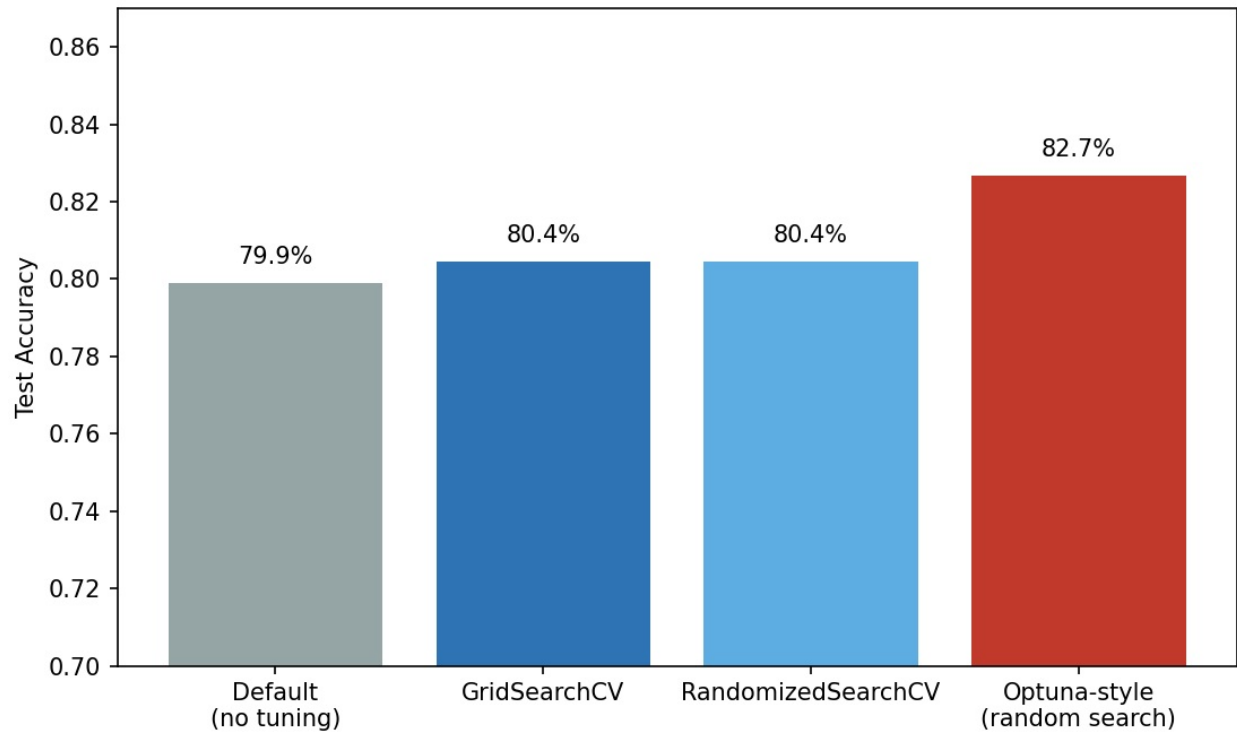
grid_search = GridSearchCV(gb_model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

random_search = RandomizedSearchCV(gb_model, param_dist, n_iter=10, cv=5, scoring='accuracy')
random_search.fit(X_train, y_train)
```

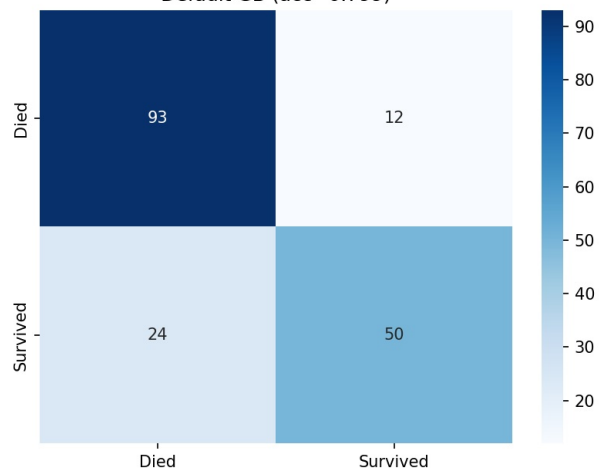
C.4 Results

Method	Best Parameters	Test Accuracy
Baseline (no tuning)	defaults	79.9%
GridSearchCV	<code>learning_rate=0.1, max_depth=3, n_estimators=200</code>	80.4%
RandomizedSearchCV	<code>learning_rate=0.03, max_depth=3, n_estimators=200</code>	80.4%
Optuna-style search (substitute)	<code>learning_rate≈0.095, max_depth=4, n_estimators=100</code>	82.7%

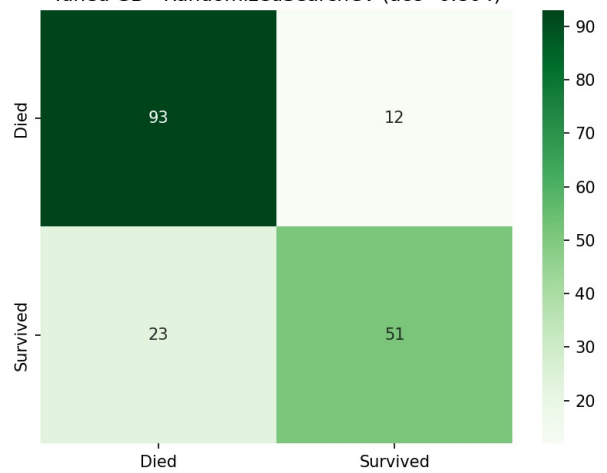
Gradient Boosting: Accuracy by Tuning Method (Titanic)

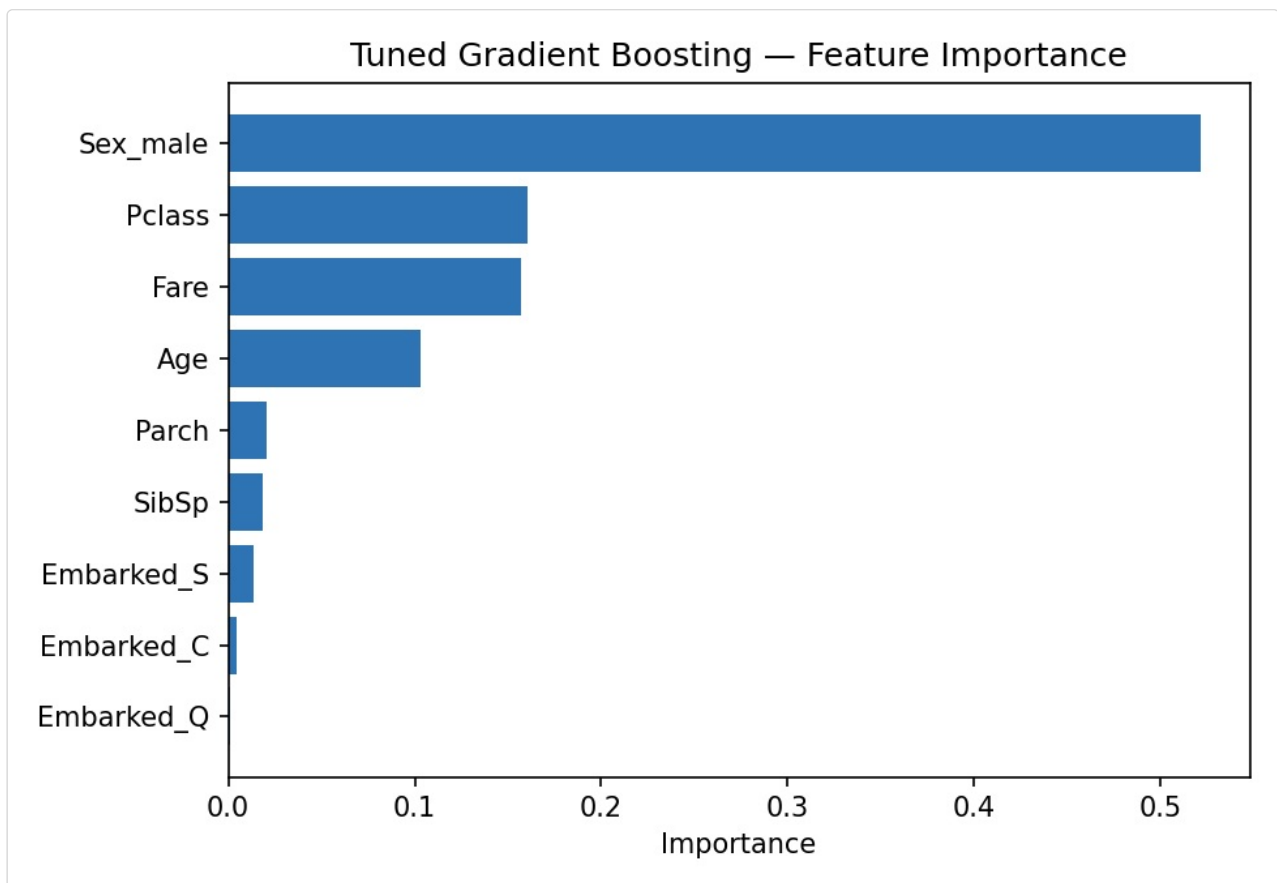


Default GB (acc=0.799)



Tuned GB - RandomizedSearchCV (acc=0.804)





All three tuning strategies improved on the untuned baseline, with the broader, more flexible random/Optuna-style search finding a meaningfully better configuration (82.7% vs. 79.9% baseline — a 2.8-point gain) than the narrower `GridSearchCV` / `RandomizedSearchCV` grids, which were constrained to fewer discrete parameter values. This illustrates a general principle: **the ceiling of any hyperparameter search is set by the breadth of the search space, not just the search algorithm** — `GridSearchCV` and `RandomizedSearchCV` here searched the same narrow grid and, unsurprisingly, landed on similar results.

Cross-Cutting Insights

- **Hyperparameter tuning's payoff depends on how much room a model has to improve.** On easy, small datasets (Iris), tuning barely moved the needle on test accuracy. On a harder, noisier, real-world dataset (Titanic), broader hyperparameter search yielded a clear, multi-point accuracy gain.
- **Tree depth is the single most consistent lever for controlling overfitting** across both the classification and regression tasks in this study — every depth sweep performed here (Iris implicitly via `GridSearchCV`, sine regression explicitly) showed the same U-shaped test-error curve.
- **Search space design matters as much as search algorithm choice** — the Titanic results show that a wider, more flexible parameter range (as used in the Optuna-style search) found a better configuration than a narrower grid searched more exhaustively.
- **Decision trees produce step-function predictions in regression** — a structurally different kind of fit than smooth models like linear regression or neural networks, worth keeping in mind when the target relationship is inherently smooth (like a sine wave).

Limitations & Next Steps

- **Optuna was substituted with manual random search** due to an offline environment; results should be re-verified with actual Optuna where available, since its Bayesian sampling typically outperforms pure random search given the same number of trials.
- **The Iris test set is small (45 samples)**, making test accuracy a noisy metric for distinguishing "good" from "great" models; the CV score used during tuning is the more reliable metric for this dataset.

- **No comparison against ensemble alternatives** (Random Forest, XGBoost/LightGBM) was included for either the classification or regression task — a natural extension given Part C already introduces Gradient Boosting.
- **Titanic preprocessing here (fillna(0)) is simpler** than the mean/median imputation used in a companion Titanic project in this portfolio — worth aligning if directly comparing accuracy figures across those two projects.

How to Run

```
pip install scikit-learn matplotlib seaborn pandas numpy optuna
jupyter notebook implementing_decisionTree_classifiers_with_scikitLearn_Answer.ipynb
jupyter notebook implementing_decisionTreeRegression_using_scikitLearn_Answer.ipynb
jupyter notebook Hyperparameter_Tuning_Gradient_Boosting_Algorithm.ipynb
```

(The Gradient Boosting notebook additionally requires a Titanic `train.csv` in the working directory — see the companion Titanic decision tree project in this portfolio for the same dataset.)

Repository Contents

File	Description
<code>implementing_decisionTree_classifiers_with_scikitLearn_Answer.ipynb</code>	Part A notebook
<code>implementing_decisionTreeRegression_using_scikitLearn_Answer.ipynb</code>	Part B notebook
<code>Hyperparameter_Tuning_Gradient_Boosting_Algorithm.ipynb</code>	Part C notebook
<code>train.csv</code>	Titanic dataset (required for Part C)
<code>iris_confusion_matrices.png</code> , <code>iris_tree.png</code>	Part A visuals
<code>regression_fit.png</code> , <code>regression_depth_mse.png</code> , <code>regression_tree.png</code>	Part B visuals
<code>gb_accuracy_comparison.png</code> , <code>gb_confusion_matrices.png</code> , <code>gb_feature_importance.png</code>	Part C visuals
<code>README.md</code>	This report

Conclusion

This three-part study covers the full arc of tree-based modeling: fitting a classifier and a regressor with scikit-learn's decision trees, then layering on systematic hyperparameter search for a more powerful ensemble method (Gradient Boosting). The results show that tuning's value is highly dataset-dependent — negligible on an easy dataset like Iris, but worth 2–3 accuracy points on a noisier real-world problem like Titanic survival — and that search space breadth matters as much as the choice of search algorithm. Together with the depth-sweep analysis in Part B, this project demonstrates a solid, practical understanding of the bias-variance tradeoff that governs every tree-based model in this portfolio.

Tech stack: Python · scikit-learn · pandas · numpy · matplotlib · seaborn